

Cosmolunch talk, 2024-03-15

Vasilii Pustovoit

March 15, 2024

Plan

1. Setup of the problem
 - (a) Cosmological ICs
 - (b) Need for zoom-ins
2. Generate density perturbations
 - (a) Theoretical intro
 - i. Density perturbations δ
 - ii. Power spectrum P , Transfer function T
 - iii. PS from white noise μ
 - iv. FFT of PS vs convolution
 - v. Real-space interpretation of convolution (propagator of perturbation)
 - vi. Why convolution (pros against FFT): see Fig. 1; nested grids are easier
 - (b) Real-space transfer function
 - i. Only FFT calculated with Boltzmann solver
 - ii. Centered at 0, with certain discretization
 - iii. Real function aperiodicity is perfect
 - (c) Generating a nested initial noise field: nested grids
 - i. Explain the grid setup, Ω_s
 - ii. Nyquist wave number: smallest wavenumber that can be sampled
 - iii. Hofmann-Ribak Algorithm
 - iv. Hofmann-Ribak for nested grids
 - (d) Performing convolution
 - i. Generating convolution kernels
 - ii. Noise convolution: finest refinement lvl
 - iii. Noise convolution: further refinement lvls
3. Initial particle positions and velocity fields
 - (a) LPT
 - i. How to get x, v with LPT
 - ii. $L(q)$ vs $\phi(q, t)$
 - iii. Poisson's equationSolving Poisson's eqn on a single grid
 - i. Poisson's equation approximation
 - ii. Defining a hierarchical grid
 - iii. Restriction and injection operators
 - iv. FAS method
 - (b) Adaptive mult-grid
 - i. Extending the hierarchy of the grid
 - ii. Restrictions on I_ℓ^ℓ operators
 - iii. Polynomial interpolation on the boundary
 - iv. Flux conservation
 - v. Near-Nyquist suppression

1 Generating density perturbations

These notes were mostly copied from [1].

1.1 Computing the seed density field

Assume that there is a power spectrum that we want to replicate.

$$P(k) = \langle \tilde{\delta}(k) \tilde{\delta}(k) \rangle \quad (1)$$

Where tilde denotes a FT, and δ is the overdensity. One could represent such a power spectrum with a transfer function:

$$P(k) = \alpha k^{n_s} \mathcal{T}^2(k) \quad (2)$$

Here α is the normalization constant, and n_s is the primordial (post-inflation) power spectrum slope.

Let's start with generating with the white noise field $\mu(r)$.

In order to make these values follow a power spectrum, we can, in Fourier space, multiply it by the power spectrum:

$$\tilde{\delta}(k) = \sqrt{P(|k|)} \tilde{\mu}(k) = \alpha |k|^{n_s/2} \mathcal{T}(|k|) \tilde{\mu}(k) \quad (3)$$

And δ in real space is obtained via IFFT.

Another way to think about this is:

$$\delta(\mathbf{r}) = T(|\mathbf{r}|) \star \mu(\mathbf{r}) \quad (4)$$

Where $\star$ corresponds to convolution, and $T(r)$ is the IFT of $\mathcal{T}$

Think about $T(k)$ as a "propagator" of a single Gaussian white noise fluctuation $\mu_q(x) \equiv \delta_D(x - q)$, where δ_D is the delta function.

There are big differences between these two approaches.

- FT forces periodicity in the real space
- 2-point correlation function is underestimated in the FT as a result of periodicity (especially in small boxes)
- It's easier to treat nested grids in convolutional approach

The convolution perturbation imprints the density perturbations due to $\mu(x)$ onto the real space.

Goal: develop a method to overlap the "propagator" with coarse-fine boundaries for nested grids.

1.2 The real-space transfer function

$\mathcal{T}$ is usually computed via a Boltzmann solver

$$T_{\mathcal{R}}(r) = \frac{1}{(2\pi)^3} \int_{\mathbb{R}} \tilde{T}(k) \exp(i \mathbf{x} \cdot \mathbf{k}) d^3k \quad (5)$$

$$= \frac{4\pi}{(2\pi)^3} \int_0^\infty \tilde{T}(k) \frac{\sin(kr)}{kr} k^2 dk. \quad (6)$$

Where $T_{\mathcal{R}}(r)$ is the real-space transfer function. The resulting transform is highly accurate over many orders of magnitude.

Finally, the convolution in eq. (4) has to be computed numerically. For that, we center at the origin, and compute the discretization $T(\mathbf{x}_{ijk}) = \mathcal{T}_{\mathcal{R}}(|\mathbf{x}_{ijk}|)$, $\mathbf{x}_{ijk} = (ih, jh, kh)$, $(i, j, k) \in [-N/2 + 1, N/2]^3$. Then, white noise is created over the same discretization, then both T and μ are FFT'd. The IFFT of the result is δ , a.k.a. overdensities.

They assume real periodically replicated transfer function. This is not the same as FFT, since (I assume) no periodicity is stated for μ . Also, it is sampling of real-space function T instead of k-sampling of $\mathcal{T}$. In actual applications they suggest using the truncated (aperiodic) transfer function.

Here's the comparison between them 1:

The drop in periodic functions is mainly to additional integral constant.

$$0 = \int_{\mathcal{D}} \xi(\mathbf{x}) d^3x \simeq 4\pi \int \xi(r) r^2 dr, \quad (7)$$

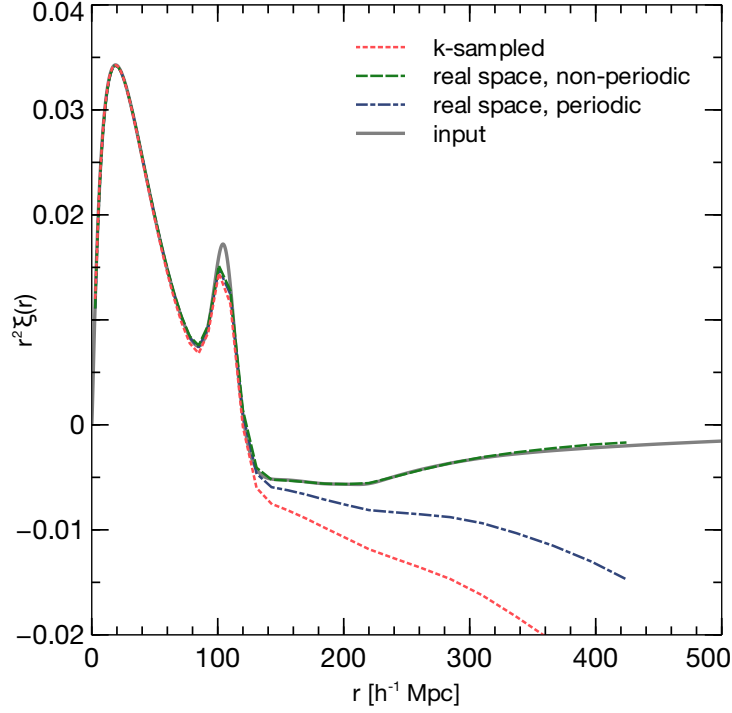


Figure 1: Auto-correlation function of the transfer function (equivalent to the two-point correlation function) for k -space sampled (dotted red) and real space transfer functions that are non-periodic (dashed green) and periodic on the box length (dot-dash blue). The gray line indicates the two-point correlation function determined from the input power spectrum at $z = 45$. All auto-correlation functions have been computed for a $500 h^{-1}\text{Mpc}$ box with 256^3 resolution.

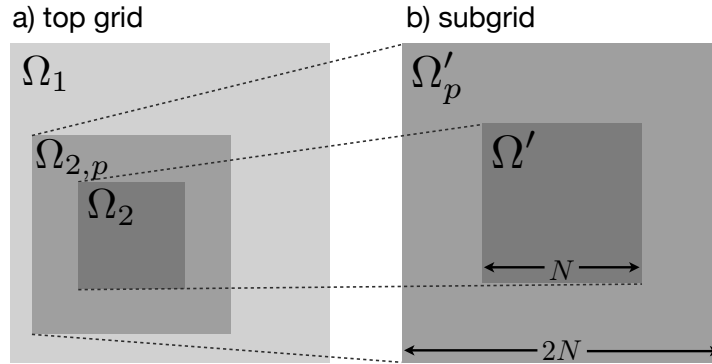


Figure 2: Schematic representation of a set-up with one refinement level. The left panel a) shows the top grid Ω which consists of a region Ω_2 covered by a refinement grid and the non-refined part $\Omega \setminus \Omega_2$. The right panel b) shows the sub-grid domain Ω' which is equivalent to Ω_2 in the left panel but has twice the resolution. To this sub-grid a padding region Ω'_p will be added when performing the FFT convolution with isolated boundaries and is denoted by $\Omega_{2,p}$ on the top grid.

Figure 3

1.3 Generating a nested initial density field

One must take into account mass conservation, in order to not alter the average density of the Universe.

(Explain the structure of the grid, Ω s, etc.)

k_{Ny} is the Niquist wave number, the smallest wavenumber that can be sampled with our simulation, it can be found as:

$$k_{Ny} = \frac{\pi}{\Delta x} \quad (8)$$

Hofmann-Ribak Algorithm:

1. First an unconstrained white noise field $\omega^{\ell+1}$ is generated for level $\ell + 1$ which has 8 times the variance of the noise field for level ℓ in the case of a refinement factor of 2.
i.e. $\sigma(\omega^{\ell+1}) = 2^D \cdot \sigma(\omega^\ell)$, where D is dimension of the sim
2. Next, for each group of eight children cells on the fine grid, the mean is replaced by the respective value of the noise field on the coarser level ℓ .
i.e. $\bar{\mu}^{\ell+1} = \bar{\mu}^\ell$

Or, in short:

1. $\sigma(\omega^{\ell+1}) = 2^D \cdot \sigma(\omega^\ell)$
2. $\bar{\mu}^{\ell+1} = \bar{\mu}^\ell$

This however doesn't retain the Fourier modes. In order to do that, we would need to modify the algorithm:

1. First an unconstrained white noise field $\omega^{\ell+1}$ is generated for level $\ell + 1$ which has 8 times the variance of the noise field for level ℓ in the case of a refinement factor of 2.
i.e., $\sigma(\omega^{\ell+1}) = 2^D \cdot \sigma(\omega^\ell)$, where D is the dimension of the simulation
2. Perform a Fourier Transform (FFT) on both the coarse grid Ω_2 and the fine grid Ω' to obtain their Fourier coefficients:
 $\tilde{\mu}_2 = \text{FFT}(\mu_2)$ and $\tilde{\mu}' = \text{FFT}(\mu')$
3. In the Fourier space of the fine grid, replace all modes with wavenumber $k \leq k_{Ny,\ell}$ (up to the Nyquist wavenumber of the coarse grid) with the Fourier coefficients from the coarse grid:
$$\tilde{\mu}'(k) = \begin{cases} \tilde{\mu}_2(k), & \text{if } k \leq k_{Ny,\ell} \\ \tilde{\mu}'(k), & \text{otherwise} \end{cases}$$
4. Perform an Inverse Fourier Transform (IFFT) on the modified fine grid to obtain the spatial grid with preserved coarse grid structures and added fine grid details:
 $\mu' = \text{IFFT}(\tilde{\mu}')$
5. Apply the reversed Hoffman-Ribak algorithm: replace the coarse grid white noise values with the average over the eight children cells of the fine grid within the refinement region to ensure mass conservation:
$$\omega_{\text{coarse}}^\ell = \frac{1}{2^D} \sum_{i=1}^{2^D} \omega_{\text{fine},i}^{\ell+1}$$
6. Ensure that the mass of the entire subgrid is conserved by verifying that the sum of the fine grid densities equals the parent coarse grid density:
$$\sum_{\text{fine}} \rho_{\text{fine}} = \rho_{\text{coarse}}$$

Or, in short:

1. $\sigma(\omega^{\ell+1}) = 2^D \cdot \sigma(\omega^\ell)$, where D is the dimension of the simulation
2. $\tilde{\mu}_2 = \text{FFT}(\mu_2)$ and $\tilde{\mu}' = \text{FFT}(\mu')$
3.
$$\tilde{\mu}'(k) = \begin{cases} \tilde{\mu}_2(k), & \text{if } k \leq k_{Ny,\ell} \\ \tilde{\mu}'(k), & \text{otherwise} \end{cases}$$
4. $\mu' = \text{IFFT}(\tilde{\mu}')$
5. $\omega_{\text{coarse}}^\ell = \frac{1}{2^D} \sum_{i=1}^{2^D} \omega_{\text{fine},i}^{\ell+1}$
6. $\sum_{\text{fine}} \rho_{\text{fine}} = \rho_{\text{coarse}}$

For further levels, one can compute by repeating steps 2-4 for all levels at increasing resolution, followed by the correction step that replaces all coarse cells at all coarser levels with the average over the eight fine cells, starting at the finest level. In other words, the following gets repeated:

1. $\tilde{\mu}_2 = \text{FFT}(\mu_2)$ and $\tilde{\mu}' = \text{FFT}(\mu')$
2. $\tilde{\mu}'(k) = \begin{cases} \tilde{\mu}_2(k), & \text{if } k \leq k_{Ny,\ell} \\ \tilde{\mu}'(k), & \text{otherwise} \end{cases}$
3. $\mu' = \text{IFFT}(\tilde{\mu}')$

1.3.1 Generating convolution kernels

We need to generate convolution kernels $T(r)$ for all levels. They are constructed in the real space, starting from the finest level. If finest level is ℓ , then at that point the convolution kernel is just the real-space transfer function $\mathcal{T}_{\mathcal{R}}$. It is centered at the origin.

Now, to save computation time, instead of generating high-res density and sampling it at a coarser level, we can "coarsify" the convolution kernel instead at the coarser way, so that it conserves the mass:

$$T^{\ell-1}(x, y, z) = \frac{1}{8} \sum_{i,j,k \in \{-\frac{1}{2}, +\frac{1}{2}\}} T^{\ell}(x+i, y+j, z+k), \quad (9)$$

This calculation is only done for the cells inside of the refined region and its padding $\Omega_{2,p}$. The padding region has twice the length of the original (so instad of a cube with side L , it has a side of $2L$). The coarse region outside of this refined one is being sampled as just a real transfer function $T^{\ell-1} = \mathcal{T}_{\mathcal{R}}$.

This mechanism ($T^{\ell-1} = \mathcal{T}_{\mathcal{R}}$ violates the strict mass conservation, since eq. 9 doesn't necessarily hold outside of the padded region. But, since the convolution kernel is more concentrated at the center (has more "mass" at the center), these errors at the coarse regions are small.

This resampling is repeated from highest to lowest ℓ , until the coarsest level is reached.

1.3.2 Noise convolution: The first refinement level

In order to perform convolution to obtain density, we will be going from top (lowest ℓ) to bottom (highest ℓ).

For the top level, $\delta^{\ell} = T^{\ell} \star \mu^{\ell}$ is automatically satisfied by the FFT.

Now, for the refined grid, we would have a contribution of the finest grid itself (δ_{self}), the coarse grid (δ_{coarse}), and the boundary (δ_{bnd}).

1. The coarse grid contribution $\delta_{\text{coarse}}^{\ell+1}$ to the refinement region is computed:
 - (a) Zero-out μ^{ℓ} on $\Omega_2 \rightarrow \mu_{\Omega \setminus \Omega_2}^{\ell}$ (defined on the entirety of Ω)
 - (b) Compute the convolution $\delta_1^{\ell} = T^{\ell} \star \mu_{\Omega \setminus \Omega_2}^{\ell}$
 - (c) Interpolate δ_1^{ℓ} from Ω_2 to $\delta_{\text{coarse}}^{\ell+1}$ (they use tri-cubic interpolation)
2. On level $\ell + 1$, isolated boundary conditions apply, so Ω' has to be padded to twice its size $2N$ to enable FFT-based convolution for computing the self-density $\delta_{\text{self}}^{\ell+1}$ due to the sub-grid alone:
 - (a) Zero-out $\mu^{\ell+1}$ on Ω'_p to focus on $\Omega' \rightarrow \mu_{\Omega'}^{\ell+1}$ (ensuring only the high-resolution region Ω' is considered)
 - (b) Compute the convolution $\delta_{\text{self}}^{\ell+1} = T^{\ell+1} \star \mu_{\Omega'}^{\ell+1}$ (calculating the self-contribution from the refined grid's white noise)
3. To reduce boundary errors, a correction term $\delta_{\text{bnd}}^{\ell+1}$ accounts for external fluctuations:
 - (a) Reverse the Hoffman-Ribak method by subtracting coarse grid white noise from each child cell, focusing effects outside Ω'
 - (b) Zero-out $\mu^{\ell+1}$ on $\Omega' \rightarrow \mu_{\Omega'_p}^{\ell+1}$
 - (c) Despite isolated conditions, opt for periodicity over truncation to minimize errors, considering larger scale effects
 - (d) $\delta_{\text{bnd}}^{\ell+1} = T^{\ell+1} \star \mu_{\Omega'_p}^{\ell+1}$
4. Concluding with spatial refinement:
 - (a) The boundary correction's impact is refined (interpolated) onto $\Omega_{2,p}$, incorporating smaller scale fluctuations into the coarse grid (ensuring the refined simulation captures both local and adjacent influences accurately)

(b) Apply them to the coarse grid

5. Combine for high-res grid:

$$\delta^{\ell+1} = \delta_{\text{self}}^{\ell+1} + \delta_{\text{coarse}}^{\ell+1} + \delta_{\text{bnd}}^{\ell+1} \quad (10)$$

1.3.3 Noise convolution: Further refinement levels

For levels $\ell + i, i \geq 2$:

$$\delta^{\ell+i} = \delta_{\text{self}}^{\ell+i} + \delta_{\text{bnd}}^{\ell+i} + \sum_{j=0, \dots, i-1} \mathcal{P}^{i-j} [\delta_{\text{coarse}}^{\ell+j-1}], \quad (11)$$

where $\mathcal{P}^{i-j} [\cdot]$ indicates a successive interpolation between i, j levels.

2 Initial particle positions and velocity fields

2.1 Lagrangian perturbation theory

Lagrangian perturbation theory describes the evolution of density perturbations in the rest-frame of the fluid. The position $\mathbf{x}$ of a fluid element at time t with respect to its initial position $\mathbf{q}$, and the respective fluid velocity, can then be written as

$$\mathbf{x}(t) = \mathbf{q} + \mathbf{L}(\mathbf{q}, t), \quad \dot{\mathbf{x}}(t) = \frac{d}{dt} \mathbf{L}(\mathbf{q}, t) \quad (12)$$

where $\mathbf{L}(\mathbf{q})$ we call the “displacement field” which is derived using perturbation theory.

From perturbation theory, we get:

$$\mathbf{L}(\mathbf{q}) = -\frac{2}{3H_0^2 a^2 D_+(t)} \nabla_{\mathbf{q}} \phi(\mathbf{q}, t) \equiv D_+^{-1}(t) \nabla_{\mathbf{q}} \Phi(\mathbf{q}, t), \quad (13)$$

And the potential obeys the Poisson eqn:

$$\Delta_{\mathbf{q}} \phi(\mathbf{q}, t) = \frac{3}{2} H_0^2 a^2 \delta(\mathbf{q}, t). \quad (14)$$

This is for the 1st-order LPT. For the 2nd-order LPT, one can say:

$$\mathbf{L}(\mathbf{q}, t) = D_+(t) \nabla_{\mathbf{q}} \Phi(\mathbf{q}, t) + D_2(t) \nabla_{\mathbf{q}} \Psi(\mathbf{q}, t), \quad (15)$$

where Ψ obeys the Poisson equation $\Delta_{\mathbf{q}} \Psi(\mathbf{q}, t) = \tau(\mathbf{q}, t)$, with

$$\tau(\mathbf{q}, t) = -\frac{1}{2} \sum_{i,j} \left[(\partial_{q_i} \partial_{q_j} \Phi)^2 - (\partial_{q_i} \partial_{q_i} \Phi) (\partial_{q_j} \partial_{q_j} \Phi) \right], \quad (16)$$

and $D_+(t)$ is the growth factor of linear perturbations, and $D_2(t) \simeq \frac{3}{7} D_+^2(t)$.

2.2 Multi-grid solution of Poisson’s equation

Both first and second order Lagrangian perturbation theory for velocity and displacement fields require the numerical solution of Poisson’s equation followed by calculating gradients. This can be achieved by use of the multi-grid algorithm.

Setup: we need to solve the equation:

$$\Delta \phi(\mathbf{x}) = f(\mathbf{x}) \quad \text{on domain } \Omega, \quad (17)$$

with periodic boundary conditions. We cover Ω with a hierarchy of grids $M^0, M^1, \dots, M^m$ of grid spacing h^i, i taking appropriate values. $h^\ell/h^{\ell+1} = 2$. m is the lowest level of our simulation grid. We will be trying to obtain lowest-level solution to Poisson eqn (lowest in a sense that lowest-res grid that we will be using in the final ICs).

$$\mathbf{L}^\ell u^\ell(x) = f^\ell(x) \quad \text{for } x \in M^\ell, \quad (18)$$

Full Approximation Storage (Scheme) (FAS) is used to solve the Poisson equation on a grid. Define $\mathbf{I}_\ell^{\ell-1}$ as the restriction and $\mathbf{I}_\ell^{\ell+1}$ as the injection operator.

Then FAS algorithm is applied as:

1. **Initial Condition for the Coarsest Level:** If $\ell = 0$, set $u^0 \equiv 0$. This step initializes the solution at the coarsest level of the grid hierarchy.

2. **Pre-Smoothing:** Apply ν_1 smoothing steps, $u_i^\ell \equiv S(u_{i-1}^\ell, f^\ell)$, $i = 1 \dots \nu_1$. Smoothing is aimed at reducing high-frequency errors before further processing.
3. **Calculate the Residual:** Calculate the residual, $r^\ell \equiv f^\ell - L^\ell u^\ell$. The residual measures how far the current solution is from satisfying the Poisson equation.
4. **Restrict the Residual:** Restrict the residual, $r^{\ell-1} \equiv I_\ell^{\ell-1} r^\ell$. This transfers the residual information to a coarser grid for broader correction.
5. **Restrict the Smoothed Solution:** Restrict the smoothed solution, $u^{\ell-1} \equiv I_\ell^{\ell-1} u_{\nu_1}^\ell$. This prepares the smoothed solution from the finer grid for correction at the coarser level.
6. **Apply the τ -Correction:** Apply the τ -correction, $f^{\ell-1} \equiv r^{\ell-1} + L^{\ell-1} u^{\ell-1}$. This adjusts the source term at the coarser level using the residual and smoothed solution from the finer level.
7. **Recursive Application of FAS:** Apply the FAS scheme recursively to solve $L^{\ell-1} u^{\ell-1} = f^{\ell-1}$. This involves solving Poisson's equation at the coarser level with the adjusted source term, then iterating this process moving up the grid levels.
8. **Correct the Solution:** Correct the solution $u^\ell \equiv u_{\nu_1}^\ell + I_{\ell-1}^\ell (u^{\ell-1} - I_\ell^{\ell-1} u_{\nu_1}^\ell)$. The corrected solution incorporates adjustments from the coarser grid.
9. **Post-Smoothing:** Apply ν_2 smoothing steps, $u_i^\ell \equiv S(u_{i-1}^\ell, f^\ell)$, $i = 1 \dots \nu_2$. This final step further refines the solution by reducing residual errors.

To sum up: start with some solution at the finest grid, smooth it out, then find out how far it is from the desired solution (find residuals). Extrapolate the solution and residuals to the coarser grid, and include the residuals as a "source term correction". This is needed to align the coarser grids more with the finer grids, "enriching" the former with the details from the latter.

2.3 Adaptive multi-grid

We will now describe the extension of the FAS multi-grid algorithm described above to additional nested adaptive grids, $M'^{m+1}, \dots, M'^{m+k}$, covering non-coextensive subdomains Ω'^i , $i = 1, \dots, k$ with $\Omega'^{i+1} \subset \Omega'^i$.

We would have to change some things about FAS to apply it to our higher-res grids:

1. Restriction $I_\ell^{\ell-1}$ and prolongation $I_{\ell-1}^\ell$ only operate on overlapping regions $\Omega'^i \cap \Omega'^{i+1}$ of the domains. The remainder $\Omega'^i \setminus \Omega'^{i+1}$ is treated as if it resided at the finest level.
2. Poisson's equation on additional sub-grids is solved with the coarse grid solution u^ℓ acting as a boundary condition for the finer level Poisson equation $L^{\ell+1} u^{\ell+1} = f^{\ell+1}$. The boundary condition is realised by adding ghost zones to the sub-grids M' to which boundary values are interpolated.

For these coarse-fine boundaries, we first use polynomial interpolation using only coarse grid information parallel to the fine grid surface. Then we interpolate in direction, normal to the surface, to find the estimated solution at the boundary (weak Dirichlet BC).

We then should constrain the fluxes through the coarse-fine surface to be conserved. This procedure can be easily understood by rewriting Poisson's equation in the following way:

$$\Delta \phi(\mathbf{x}) = \nabla \cdot \nabla \phi(\mathbf{x}) = f(\mathbf{x}) \quad (19)$$

can be integrated over one grid cell (in one dimension for notational simplicity) to yield:

$$\int_{x_i-h/2}^{x_i+h/2} \nabla \cdot \nabla \phi(\mathbf{x}) d\mathbf{x} = \int_{x_i-h/2}^{x_i+h/2} f(\mathbf{x}) d\mathbf{x} \quad (20)$$

$$\int_{x_i-h/2}^{x_i+h/2} \partial_x F_\phi(x') dx' = m_i, \quad (21)$$

where $m_i = \int f(x_i) dx'$ is the mass contained in cell i , and $F_\phi \equiv \partial_x \phi$ is the potential flux. Applying the divergence theorem, we find that

$$F_\phi(x_i + h/2) - F_\phi(x_i - h/2) = m_i, \quad (22)$$

i.e. the mass in cell i generates a flux through the cell surfaces given by F_ϕ .

Note that this flux matching can be thought of as constraints on the interpolation scheme. We thus obtain mixed boundary conditions: a von Neumann boundary condition due to the flux matching, which constrains only one degree of freedom for the boundary cells at a single coarse-fine boundary, and a subordinate Dirichlet boundary condition, which constrains all degrees of freedom, but is only used to evaluate the fluxes on the fine grid

2.4 Recovering small-scale power

Both the use of a transfer function evaluated in real space and finite difference methods lead to a loss of power at scales close to the Nyquist wavenumber. We outline methods to solve this problem in what follows. Most of these methods will lead to spectral leakage and the associated spurious oscillations which poses no problem in the case of dark matter particle initial conditions but should be avoided for baryons.

2.4.1 Finite Volume Correction

The multi-grid method is a finite volume approach, leading to a piecewise constant approximation to the source field f . To restore the small-scale fluctuation amplitudes, we perform a deconvolution with the cell assignment function, equivalent to the nearest grid point (NGP) assignment, which is described by the Fourier-transformed kernel $\widetilde{W}_s(\mathbf{k})$. The deconvolution in the Fourier domain recovers some of the sub-grid power.

2.4.2 A hybrid Poisson solver

A hybrid solver that combines finite difference methods with a k -space exact solution can recover small-scale power that is attenuated by finite difference operators. On the finest grid, we replace the solution with one obtained using the exact k -space method and correct it for the self-gravity due to the fine grid.

2.4.3 Averaging correction

Even with the hybrid solver, grid assignment of the real-space transfer function can lead to damping at small scales. We can restore this sub-grid power by averaging over sub-grid scales using a kernel K_{sg} and deconvolving with its Fourier transform $\widetilde{K}_{\text{sg}}$ to recover the power at large k values.

3 Baryon Initial Conditions

In the generation of initial conditions for a two-component fluid – dark matter and baryons – several factors differ from treating a single-component dark matter fluid:

- Distinct amplitudes of density and velocity fluctuations for baryons and CDM due to the different natures of the two components, especially evident at high redshifts.
- Exponential suppression of baryon fluctuations beyond the photon diffusion scale, altering the baryon transfer function shape on small scales.
- The necessity of respecting differences in initial velocities between the two components to accurately reproduce the growth of density perturbations.
- The importance of reflecting these differences in simulation initial conditions to match predictions from linear perturbation theory.

For density and velocity transfer functions, we account for:

- The divergence of the different transfer functions for baryon and CDM density perturbations, particularly below scales of $\sim 10^{-2}h^{-1}\text{Mpc}$.
- The use of Boltzmann solvers to obtain the velocity perturbations from relativistic perturbation theory for both fluids, leading to two separate velocity transfer functions, $T_{v,c}(k)$ and $T_{v,b}(k)$.
- The construction of real-space transfer functions for density and velocity perturbations for both baryons and CDM, allowing for the application of 2LPT.

3.1 Local Lagrangian approximation for the baryon density field

The Local Lagrangian approximation (LLA) ensures consistency between the initial baryon and dark matter density fields by:

- Employing the LLA to generate a non-Gaussian initial baryon density field consistent with the displacements from LPT.
- Correcting for non-conservation of mass due to the approximation, which typically has a negligible error at high initial redshifts.

- Providing consistent initial conditions for both baryon density fields and dark matter when using 1LPT or 2LPT.

This approach leads to a baryon density probability distribution function (PDF) less sensitive to the starting redshift of the simulation and consistent with the skewness expected from LPT.

References

- [1] Oliver Hahn and Tom Abel. “Multi-scale initial conditions for cosmological simulations: Multi-scale initial conditions”. In: *Monthly Notices of the Royal Astronomical Society* 415.3 (2011), 2101–2121. ISSN: 0035-8711. DOI: 10.1111/j.1365-2966.2011.18820.x. arXiv: 1103.6031. URL: <http://dx.doi.org/10.1111/j.1365-2966.2011.18820.x>.